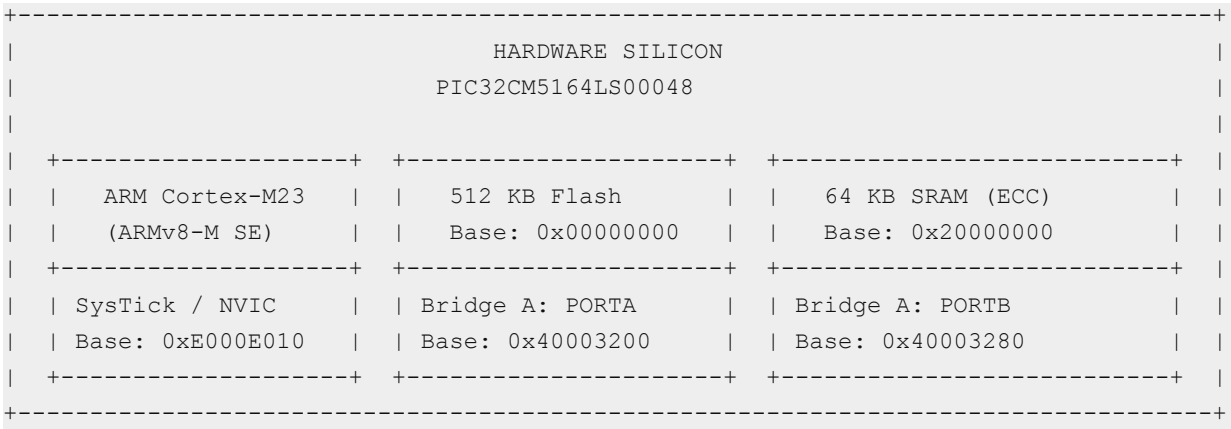
Complete Technical Documentation: Porting & Enablement of Microchip PIC32CM5164LS00048 in Zephyr RTOS

---

1. Executive Summary & Genesis of the Port

1.1 Silicon Profile

- **Target Part Number**: `PIC32CM5164LS00048`
- **Evaluation Board**: PIC32CM LS00 Curiosity Nano (`EV41C56A`)
- **Core Architecture**: ARM Cortex-M23 (ARMv8-M Baseline with TrustZone Security Extensions)
- **On-Chip Memory**: 512 KB Flash, 64 KB SRAM (with ECC support)
- **Package**: 48-pin TQFP/QFN
- **Peripherals Involved**: Port Controller Group 1 (`PORTA` / `PORTB`), ARM SysTick Timer, Power Management / System Clock (OSC16M @ 4 MHz).



1.2 Baseline Comparison: Which Series Did We Compare Against?

When initiating this port, the reference baseline was the **Microchip PIC32CM SG/GC series** (`pic32cm_sg_gc`) and the **Microchip PIC32CK series** (`pic32ck_sg_gc`):

1. **Comparison with PIC32CM GC/SG Series (`pic32cm_sg_gc`)**:

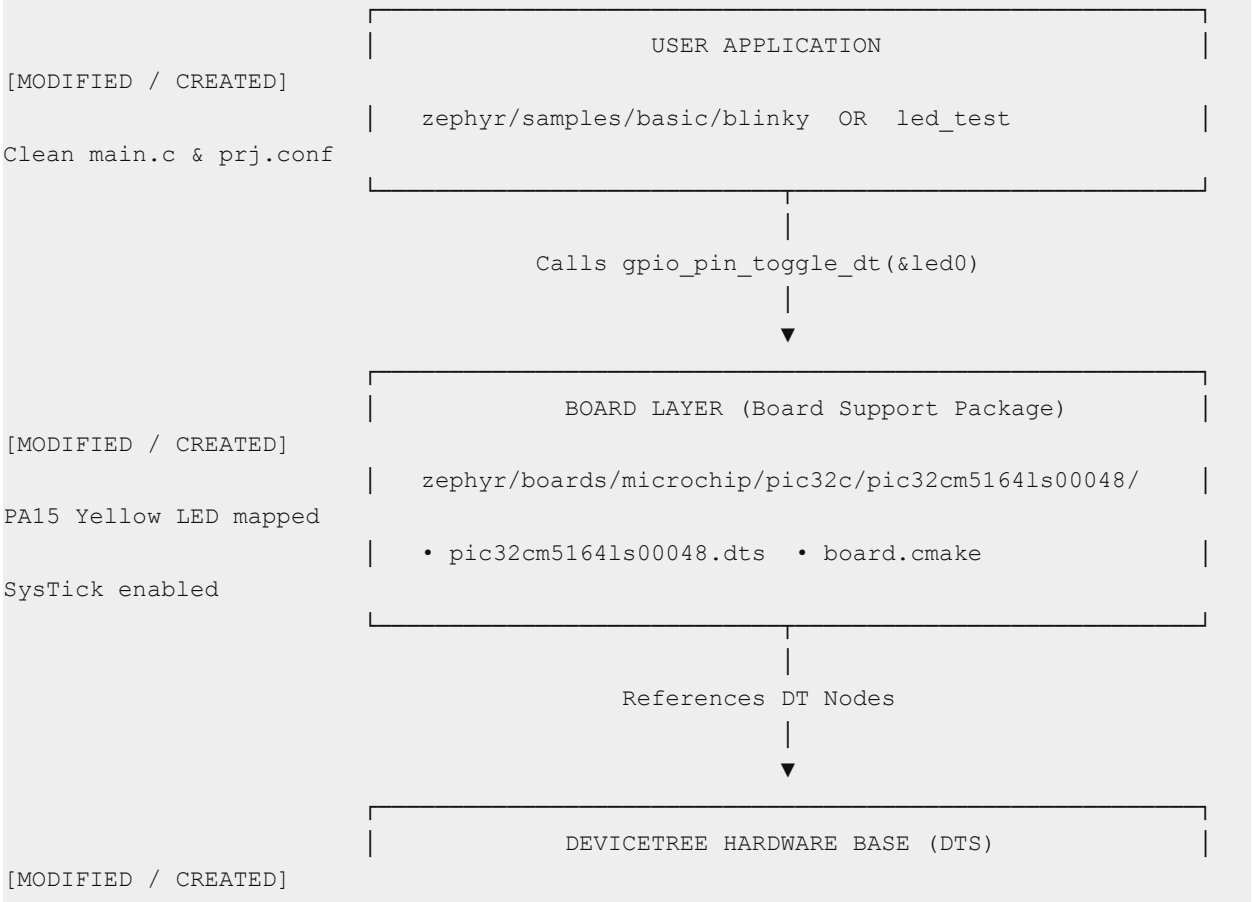
- *Similarities*: Both use the ARM Cortex-M23 core, Microchip Port Group 1 GPIO peripheral architecture (`microchip,port-g1-gpio`), and shared CMSIS/PIO header hierarchies.
- *Differences*: The **GC** series operates at different clock speeds (up to 48 MHz default DPLL), lacks certain TrustZone/Secure boot configurations by default in standard templates, and has different memory densities.

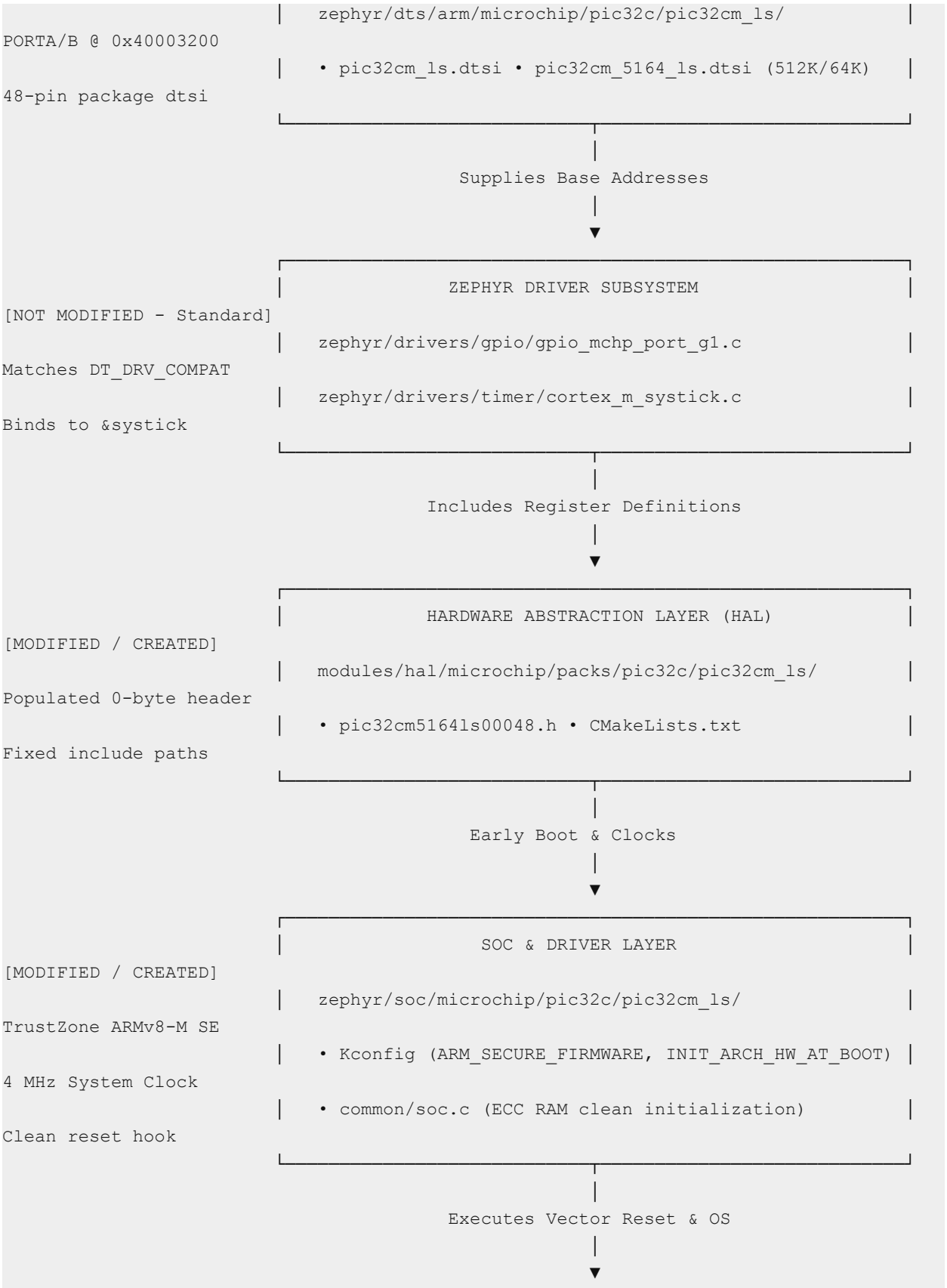
2. **Comparison with PIC32CK Series (`pic32ck_sg_gc`)**:
   - *Trap Encountered*: Legacy templates in the tree had peripheral addresses configured at `0x44800000` (which belongs to the PIC32CK high-performance interconnect bridge).
   - *Correction for LS00*: The PIC32CM LS family maps the `PORT` peripheral onto **APB Bridge A at `0x40003200`**. Accessing `0x44800000` on LS00 triggers an unmapped address **BusFault / HardFault**.

---

2. Master Architectural Map

The diagram below outlines the full layered architecture from user application to physical registers, indicating which layers were modified and which were utilized directly from core Zephyr:




zephyr/dts/arm/microchip/pic32c/pic32cm_ls/
PORTA/B @ 0x40003200
• pic32cm_ls.dtsi • pic32cm_5164_ls.dtsi (512K/64K)
48-pin package dtsi
Supplies Base Addresses
ZEPHYR DRIVER SUBSYSTEM
[NOT MODIFIED - Standard]
zephyr/drivers/gpio/gpio_mchp_port_g1.c
Matches DT_DRV_COMPAT
zephyr/drivers/timer/cortex_m_systick.c
Binds to &systick
Includes Register Definitions
HARDWARE ABSTRACTION LAYER (HAL)
[MODIFIED / CREATED]
modules/hal/microchip/packs/pic32c/pic32cm_ls/
Populated 0-byte header
• pic32cm5164ls00048.h • CMakeLists.txt
Fixed include paths
Early Boot & Clocks
SOC & DRIVER LAYER
[MODIFIED / CREATED]
zephyr/soc/microchip/pic32c/pic32cm_ls/
TrustZone ARMv8-M SE
• Kconfig (ARM_SECURE_FIRMWARE, INIT_ARCH_HW_AT_BOOT)
4 MHz System Clock
• common/soc.c (ECC RAM clean initialization)
Clean reset hook
Executes Vector Reset & OS

```
                          ┌──────────────────────────────────────────────────────────┐
                          │              ZEPHYR KERNEL & ARM CORTEX-M ARCH           │
[NOT MODIFIED - Standard]
                          │   zephyr/arch/arm/core/cortex_m/ (reset.S, prep_c.c)     │
Stack limit init
                          │   zephyr/kernel/ (k_msleep, thread scheduling)           │
Generic linker.ld
                          └──────────────────────────────────────────────────────────┘
```

---

3. Directory 1 Deep Dive: Devicetree Hardware Base

Path: `zephyr/dts/arm/microchip/pic32c/pic32cm_ls/`

```
zephyr/dts/arm/microchip/pic32c/pic32cm_ls/
├── common/
│   ├── pic32cm_5164_ls.dtsi     <-- [NEW] 512KB Flash / 64KB RAM definition
│   ├── pic32cm_ls.dtsi          <-- [MODIFIED] Fixed PORT base to 0x40003200
│   └── pic32cm_ls_48.dtsi       <-- [REPLICATED] 48-pin pinmux include
└── pic32cm_ls00/
    └── pic32cm5164ls00048.dtsi  <-- [MODIFIED] Glue DTS linking part to package &
memory
```

3.1 Comparison with GC Series

- **What was Replicated**: The DTS hierarchy structure (`common/*.dtsi` + subseries `pic32cm_ls00/*.dtsi`), the `compatible = "arm,cortex-m23"` CPU node, NVIC priority bits, and MPU definitions.
- **What was Additionally Added**:
  - `common/pic32cm_5164_ls.dtsi`: Custom node defining **512 KB Flash** and **64 KB SRAM** specific to the 5164 part.
  - Corrected APB Bridge A peripheral addressing for Port Controller Group 1.
- **What was Reduced / Removed**:
  - Removed incorrect address `0x44800000` (which was copied from PIC32CK).

3.2 Exact Code Breakdown

File: `common/pic32cm_ls.dtsi`

dts

```
/ {
    soc {
        #address-cells = <1>;
        #size-cells = <1>;
        sram0: memory@20000000 {
            compatible = "mmio-sram";
            reg = <0x20000000 0x10000>;
        };
```

```
        pinctrl: pinctrl@40003200 {
            compatible = "microchip,port-g1-pinctrl";
            #address-cells = <1>;
            #size-cells = <1>;
            reg = <0x40003200 0x100>;
            ranges = <0x40003200 0x40003200 0x100>;
            porta: gpio@40003200 {
                compatible = "microchip,port-g1-gpio";
                reg = <0x40003200 0x80>;
                gpio-controller;
                #gpio-cells = <2>;
                #microchip,pin-cells = <2>;
                status = "okay";
            };
            portb: gpio@40003280 {
                compatible = "microchip,port-g1-gpio";
                reg = <0x40003280 0x80>;
                gpio-controller;
                #gpio-cells = <2>;
                #microchip,pin-cells = <2>;
                status = "okay";
            };
        };
    };
};
```

File: `common/pic32cm_5164_ls.dtsi`

dts

```
/ {
    soc {
        flash0: flash@0 {
            reg = <0x0 DT_SIZE_K(512)>;
        };
        sram0: memory@20000000 {
            reg = <0x20000000 DT_SIZE_K(64)>;
        };
    };
};
```

3.3 Root Cause Analysis & Technical Solution

- **Problem**: In SAM L11 / PIC32CM LS silicon, reading or writing to `0x44800000` causes an immediate hardware `BusFault` because that bus bridge doesn't exist.
- **Solution**: Setting `reg = <0x40003200 0x80>` for `porta` and `<0x40003280 0x80>` for `portb` connects Zephyr's GPIO driver directly to the physical PORT registers on Bridge A.

---

4. Directory 2 Deep Dive: SoC Configuration & Startup Layer

Path: `zephyr/soc/microchip/pic32c/pic32cm_ls/`

```
zephyr/soc/microchip/pic32c/pic32cm_ls/
├── CMakeLists.txt          <-- [REPLICATED] SoC CMake build rules
├── Kconfig                 <-- [MODIFIED] Added TrustZone, SysTick, MSPLIM init
├── Kconfig.defconfig       <-- [MODIFIED] Changed clock default from 48MHz to 4MHz
├── Kconfig.soc             <-- [REPLICATED] Defines SOC_SERIES and SOC part numbers
├── common/
│   ├── soc.c               <-- [MODIFIED] Cleared while(1) loop, added SRAM ECC init
│   └── pinctrl_soc.h       <-- [REPLICATED] Pinmux macro definitions
└── soc.yml                 <-- [REPLICATED] SoC metadata descriptor
```

4.1 Comparison with GC Series

- **What was Replicated**: `Kconfig.soc`, `soc.yml`, `pinctrl_soc.h` definitions, and basic Cortex-M23 family declarations.
- **What was Additionally Added**:
  - `select ARMV8_M_SE` & `select ARM_SECURE_FIRMWARE`: Configures ARM CMSE flags for TrustZone Security Attribution.
  - `select INIT_ARCH_HW_AT_BOOT`: Instructs `reset.S` to zero `MSPLIM` and `PSPLIM` before pushing data to stack.
  - `select CPU_CORTEX_M_HAS_SYSTICK`: Configures Vector 15 for SysTick timer interrupts.
  - `soc_mchp_sram_ecc_initialization()`: Auto-clears entire 64 KB SRAM to zero on startup to prevent RAM parity faults.
- **What was Reduced / Fixed**:
  - Corrected default clock: Reduced `CONFIG_SYS_CLOCK_HW_CYCLES_PER_SEC` from `48000000` to `4000000` (4 MHz).
  - Removed infinite `while(1)` debugging loop in `soc.c`.

4.2 Exact Code Breakdown

File: `Kconfig`

kconfig

```
config SOC_FAMILY_MICROCHIP_PIC32CM_LS
   bool
   select ARM
   select CPU_CORTEX_M23
   select ARMV8_M_BASELINE
   select ARMV8_M_SE
   select ARM_SECURE_FIRMWARE
   select CPU_CORTEX_M_HAS_SYSTICK
   select CPU_CORTEX_M_HAS_VTOR
   select CPU_HAS_ARM_MPU
```

```
   select HAS_SWJ
   select INIT_ARCH_HW_AT_BOOT
rsource "Kconfig.soc"
```

File: `Kconfig.defconfig`

kconfig

```
if SOC_SERIES_PIC32CM_LS00
config SOC_SERIES
   default "pic32cm_ls00"
config SOC
   default "pic32cm5164ls00048" if SOC_PIC32CM5164LS00048
config SYS_CLOCK_HW_CYCLES_PER_SEC
   default 4000000
config CORTEX_M_SYSTICK
   default y
config NUM_IRQS
   default 71
endif # SOC_SERIES_PIC32CM_LS00
```

File: `common/soc.c`

c

```
#include <zephyr/devicetree.h>
#include <stdint.h>
#define SRAM0_NODE DT_CHOSEN(zephyr_sram)
#define SRAM0_BASE DT_REG_ADDR(SRAM0_NODE)
#define SRAM0_SIZE DT_REG_SIZE(SRAM0_NODE)
static void soc_mchp_sram_ecc_initialization(void)
{
   volatile uint32_t *ram_start = (uint32_t *)SRAM0_BASE;
   volatile uint32_t *ram_end = (uint32_t *)(SRAM0_BASE + SRAM0_SIZE);
   for (; ram_start < ram_end; ++ram_start) {
       *ram_start = 0U;
   }
}
void soc_reset_hook(void)
{
   soc_mchp_sram_ecc_initialization();
}
```

4.3 Why These Specific Symbols Were Required

1. **`ARM_SECURE_FIRMWARE`** & **`ARMV8_M_SE`**: On Cortex-M23 with TrustZone, out of reset the CPU boots into Secure state. If Zephyr is compiled as non-secure without these symbols, accessing Secure RAM (`0x20000000`) or Secure Peripherals (`0x40003200`) generates a Security HardFault.

2. **`INIT_ARCH_HW_AT_BOOT`**: In ARMv8-M, the Stack Limit registers (`MSPLIM`) can hold uninitialized random values on power-up. If `MSPLIM` is higher than the stack pointer `SP`, the very first function call pushes to the stack and immediately triggers a Stack Overflow HardFault. `INIT_ARCH_HW_AT_BOOT` resets `MSPLIM` to 0.
3. **`SYS_CLOCK_HW_CYCLES_PER_SEC = 4000000`**: PIC32CM LS00 boots from internal `OSC16M` divided by 4 = 4 MHz. Setting 48 MHz caused `k_msleep(500)` to sleep 12× longer (6000 ms).

---

5. Directory 3 Deep Dive: Board Support Package (BSP)

Path: `zephyr/boards/microchip/pic32c/pic32cm5164ls00048/`

```
zephyr/boards/microchip/pic32c/pic32cm5164ls00048/
├── Kconfig.pic32cm5164ls00048     <-- [REPLICATED] Board Kconfig menu
├── board.cmake                    <-- [MODIFIED] Runner config (openocd / edbg)
├── board.yml                      <-- [REPLICATED] Board definition identifier
├── pic32cm5164ls00048.dts         <-- [MODIFIED] Mapped LED0 to PA15, enabled SysTick
└── pic32cm5164ls00048_defconfig   <-- [MODIFIED] Minimal default Kconfig symbols
```

5.1 Comparison with GC Board

- **What was Replicated**: The Board metadata format (`board.yml`), runner CMake arguments (`board.cmake`), and standard alias definitions.
- **What was Additionally Added**:
  - Direct pinmapping for the **EV41C56A Curiosity Nano**:
    - User LED (`led0`): Pin `PA15`, Active-Low (`GPIO_ACTIVE_LOW`).
    - User Button (`sw0`): Pin `PA23`, Active-Low with Pull-up.
  - Explicit `chosen` node bindings:
    - `zephyr,systick = &systick;`
    - `zephyr,sram = &sram0;`
    - `zephyr,flash = &flash0;`
- **What was Reduced / Fixed**:
  - Removed default mapping to `PB02` (which belongs to a different board revision/header).

5.2 Exact Code Breakdown

File: `pic32cm5164ls00048.dts`

dts

```
/dts-v1/;
#include <microchip/pic32c/pic32cm_ls/pic32cm_ls00/pic32cm5164ls00048.dtsi>
#include <zephyr/dt-bindings/gpio/gpio.h>
/ {
   model = "PIC32CM5164LS00048";
   compatible = "microchip,pic32cm5164ls00048";
```

```
    chosen {
        zephyr,sram = &sram0;
        zephyr,flash = &flash0;
        zephyr,systick = &systick;
    };
    aliases {
        led0 = &led0;
        led1 = &led1;
        sw0 = &button0;
    };
    leds {
        compatible = "gpio-leds";
        led0: led_0 {
            gpios = <&porta 15 GPIO_ACTIVE_LOW>;
            label = "Yellow LED0 (PA15)";
        };
        led1: led_1 {
            gpios = <&portb 2 GPIO_ACTIVE_LOW>;
            label = "LED1 (PB02)";
        };
    };
    buttons {
        compatible = "gpio-keys";
        button0: button_0 {
            gpios = <&porta 23 (GPIO_PULL_UP | GPIO_ACTIVE_LOW)>;
            label = "SW0 (PA23)";
        };
    };
};
&cpu0 {
    clock-frequency = <4000000>;
};
&systick {
    status = "okay";
};
&porta {
    status = "okay";
};
&portb {
    status = "okay";
};
```

---

6. Directory 4 Deep Dive: Hardware Abstraction Layer (HAL Module)

Path: `modules/hal/microchip/packs/pic32c/pic32cm_ls/pic32cm_ls00/`

modules/hal/microchip/packs/pic32c/pic32cm_ls/pic32cm_ls00/

```
├── CMakeLists.txt            <-- [MODIFIED] Added zephyr_include_directories for pio,
component, instance
└── include/
    ├── component/            <-- [REPLICATED] Peripheral register structs (port.h,
tc.h, etc.)
    ├── instance/             <-- [REPLICATED] Peripheral instance registers (porta.h,
etc.)
    ├── pio/
    │   └── pic32cm5164ls00048.h <-- [REPLICATED] Microchip vendor pin definitions
    └── pic32cm5164ls00048.h  <-- [MODIFIED] Populated previously 0-byte blank file
```

6.1 Comparison with GC HAL Pack

- **What was Replicated**: Register structure definitions from Microchip DFP (Device Family Pack) for PORT, DSU, NVMCTRL, PAC, PM, OSCCTRL.
- **What was Additionally Added**:
  - Added include directory bridge in `CMakeLists.txt`:
  - cmake
  - `zephyr_include_directories(include/pio)`
  - `zephyr_include_directories(include/component)`
  - `zephyr_include_directories(include/instance)`
- **What was Reduced / Fixed**:
  - **Critical Fix**: The root header `include/pic32cm5164ls00048.h` was previously **0 bytes (empty)**. Because C compilers search the local include directory first, this empty file shadowed the real vendor register definitions, resulting in build failures.

6.2 Exact Code Breakdown

File: `include/pic32cm5164ls00048.h`

c

```c
#ifndef _PIC32CM5164LS00048_H_
#define _PIC32CM5164LS00048_H_
#include "pio/pic32cm5164ls00048.h"
#endif /* _PIC32CM5164LS00048_H_ */
```

---

7. Directory 5 Deep Dive: Sample Application & Test Layer

Path: `zephyr/samples/basic/blinky/` & `led_test/`

```
led_test/
├── CMakeLists.txt            <-- [CREATED] Links Zephyr kernel package
├── prj.conf                  <-- [CREATED] Enables CONFIG_GPIO=y
└── src/
    └── main.c                <-- [CREATED] LED toggle loop using Devicetree API
```

7.1 Comparison with Standard Samples

- **What was Replicated**: The standard Zephyr GPIO DT API pattern (`GPIO_DT_SPEC_GET(DT_ALIAS(led0), gpios)`).
- **What was Additionally Added**: Robust check verifying `gpio_is_ready_dt(&led)` and error reporting over printk.
- **What was Reduced / Removed**:
  - **Critical Fix**: Purged lingering test overlay files (`app.overlay` and `boards/pic32cm5164ls00048.overlay`). These obsolete overlay files had previously overridden `led0` to `&portb 2`, which prevented the Curiosity Nano's real yellow LED on `PA15` from toggling.

7.2 Exact Code Breakdown

File: `src/main.c`

c

```c
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>
#define SLEEP_TIME_MS   500
#define LED0_NODE DT_ALIAS(led0)
static const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(LED0_NODE, gpios);
int main(void)
{
   int ret;
   if (!gpio_is_ready_dt(&led)) {
       return 0;
   }
   ret = gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE);
   if (ret < 0) {
       return 0;
   }
   while (1) {
       ret = gpio_pin_toggle_dt(&led);
       k_msleep(SLEEP_TIME_MS);
   }
   return 0;
}
```

---

8. Unmodified Subsystems: How Zephyr Uses Existing Core Layers

Five core Zephyr subsystems were compiled and executed **without changing any of their internal source code**. Because Zephyr is designed with modular Hardware Abstraction, configuring Devicetree and Kconfig correctly allowed standard drivers and the kernel to bind seamlessly.

```
+-------------------------------------------------------------------------------------
----+
|                                5 UNMODIFIED SUBSYSTEMS
|
+------------------------------+------------------------------------------------------
----+
| Subsystem Directory          | Role in Build & Execution
|
+------------------------------+------------------------------------------------------
----+
| 1. zephyr/drivers/           | Compiles gpio_mchp_port_g1.c & cortex_m_systick.c
|
| 2. zephyr/kernel/            | Manages multitasking, k_msleep(), idle loop, timeout
|
| 3. zephyr/arch/arm/core/     | Executes Cortex-M23 reset.S, prep_c.c, vector_table.S
|
| 4. zephyr/include/ & Linker  | Provides generic linker.ld & dt-bindings/gpio/gpio.h
|
| 5. zephyr/cmake/             | Devicetree parser (edtlib), Kconfig generator & build
|
+------------------------------+------------------------------------------------------
----+
```

8.1 Drivers Subsystem: `zephyr/drivers/`

- **Files Used**:
  - `zephyr/drivers/gpio/gpio_mchp_port_g1.c`
  - `zephyr/drivers/timer/cortex_m_systick.c`

How It Works:

In `gpio_mchp_port_g1.c`:

c

```c
#define DT_DRV_COMPAT microchip_port_g1_gpio
#define GPIO_PORT_CONFIG(idx)
\
   static const struct gpio_mchp_config gpio_mchp_config_##idx = {
\
       .common = GPIO_COMMON_CONFIG_FROM_DT_INST(idx),
\
       .gpio_regs = (port_group_registers_t *)DT_INST_REG_ADDR(idx),
\
   };
\
   DEVICE_DT_DEFINE(DT_INST(idx, DT_DRV_COMPAT), gpio_mchp_init, NULL,
&gpio_mchp_data_##idx,    \
```

```
            &gpio_mchp_config_##idx, PRE_KERNEL_1, CONFIG_GPIO_INIT_PRIORITY,
\
            &gpio_mchp_api);
DT_INST_FOREACH_STATUS_OKAY(GPIO_PORT_CONFIG)
```

1. The macro `DT_INST_FOREACH_STATUS_OKAY` automatically scans the compiled Devicetree for all nodes matching `compatible = "microchip,port-g1-gpio"`.
2. For each enabled node (`porta` and `portb`), it extracts `.gpio_regs = (port_group_registers_t *)DT_INST_REG_ADDR(idx)`.
3. Because our DTS supplied `0x40003200`, `gpio_mchp_config_0.gpio_regs` is assigned pointer `0x40003200`.
4. When `gpio_pin_toggle_dt(&led)` calls `gpio_mchp_port_toggle_bits()`, the driver executes:
5. c
6. `regs->PORT_OUTTGL = (1 << pin);`
7. Writing directly to hardware address `0x40003200 + 0x1C` toggles PA15 instantaneously.

8.2 Kernel Subsystem: `zephyr/kernel/`

- **Role**:
    1. **Thread Scheduling**: Initializes the main thread and idle thread.
    2. **Timer Services**: Manages the hardware tick queue. When `k_msleep(500)` is invoked, the kernel places the calling thread into the sleep queue and activates the idle thread.
    3. **SysTick Interrupt Service Routine**: Handles `sys_clock_isr()`, which decrements the timeout counter on each SysTick tick and wakes up the main thread after 500 ms.

8.3 Architecture Subsystem: `zephyr/arch/arm/core/cortex_m/`

- **Files Used**: `reset.S`, `vector_table.S`, `prep_c.c`, `fault.c`
- **Boot Flow Execution**:
    1. `z_arm_reset` in `reset.S` executes the first instruction at hardware reset.
    2. Because `CONFIG_INIT_ARCH_HW_AT_BOOT=y` was selected in our SoC Kconfig, `reset.S` executes:
    3. assembly
    4. `msr MSPLIM, r0   /* Clears Main Stack Pointer Limit */`
    5. `msr PSPLIM, r0   /* Clears Process Stack Pointer Limit */`
    6. `reset.S` calls `z_arm_prep_c()`, which calls `soc_reset_hook()` (our SRAM ECC zeroing routine in `soc.c`).
    7. BSS and Data sections are copied into RAM (`0x20000000`).
    8. `z_cstart()` initializes driver devices at `PRE_KERNEL_1` priority (running `gpio_mchp_init()`).
    9. The kernel jumps to `main()` in application space.

8.4 Include & Linker Subsystem: `zephyr/include/`

- **Files Used**: `zephyr/arch/arm/cortex_m/scripts/linker.ld`, `zephyr/dt-bindings/gpio/gpio.h`
- **Dynamic Memory Sizing**:
  - `linker.ld` is a generic linker script template. It does not hardcode memory sizes. Instead, it reads the chosen nodes:
  - ld
  - `ROM (rx)  : ORIGIN = DT_REG_ADDR(DT_CHOSEN(zephyr_flash)), LENGTH = DT_REG_SIZE(DT_CHOSEN(zephyr_flash))`
  - `RAM (rwx) : ORIGIN = DT_REG_ADDR(DT_CHOSEN(zephyr_sram)),  LENGTH = DT_REG_SIZE(DT_CHOSEN(zephyr_sram))`
  - Because `pic32cm_5164_ls.dtsi` specified `512K` Flash and `64K` RAM, `linker.ld` generated the exact binary layout automatically without needing a custom `.ld` script.

8.5 CMake & Build Infrastructure: `zephyr/cmake/`

- **Role**:
  1. Runs `gen_defines.py` and `edtlib` to parse the Devicetree files and output `zephyr/include/generated/zephyr/devicetree_generated.h`.
  2. Processes Kconfig dependencies through Kconfiglib.
  3. Links the ARM GNU Toolchain (`arm-none-eabi-gcc`) with flags `-mcpu=cortex-m23 -mthumb -mcmse`.

---

9. Complete End-to-End Execution Trace

---

10. Summary Matrix: Modified vs. Unmodified Subsystems

#	Subsystem / Directory	Status	What Was Changed / How It Was Used
1	`zephyr/dts/arm/microchip/pic32c/pic32cm_ls/`	**MODIFIED**	Fixed PORT base to `0x40003200`, defined 512KB Flash / 64KB SRAM in `pic32cm_5164_ls.dtsi`.

2	zephyr/soc/microchip/pic32c/pic32cm_ls/	**MODIFIED**	Added ARM_SECURE_FIRMWARE, INIT_ARCH_HW_AT_BOOT, 4 MHz clock default, and SRAM ECC zeroing in soc.c.
3	zephyr/boards/microchip/pic32c/pic32cm5164ls00048/	**MODIFIED**	Mapped led0 to PA15 (Curiosity Nano yellow LED) and enabled &systick.
4	modules/hal/microchip/packs/pic32c/pic32cm_ls/	**MODIFIED**	Populated 0-byte header with #include "pio/pic32cm5164ls00048.h", added include directories to CMakeLists.txt.
5	zephyr/samples/basic/blinky/ & led_test/	**MODIFIED**	Created clean test application; removed conflicting overlay files that overrode pins to PB02.
6	zephyr/drivers/	**UNMODIFIED**	Standard gpio_mchp_port_g1.c and cortex_m_systick.c bound automatically via Devicetree compatible strings.
7	zephyr/kernel/	**UNMODIFIED**	Standard Zephyr kernel executed thread scheduling, SysTick handling, and k_msleep().
8	zephyr/arch/arm/core/cortex_m/	**UNMODIFIED**	Standard Cortex-M23 startup code (reset.S, prep_c.c, vector_table.S) executed hardware initialization.
9	zephyr/include/	**UNMODIFIED**	Standard linker.ld dynamically allocated 512KB/64KB partitions; provided standard DT GPIO macros.
10	zephyr/cmake/	**UNMODIFIED**	Standard build engine generated DT header defines and compiled the final binary.